Experiment No. – 12							
Date of Performance:	**3rd Aug 2026**						
Date of Submission:	**4th Aug 2026**						
Conceptual Understanding (02)	**Execution (1.5)**	**Problem Solving & Debugging(02)**	**Professional Conduct & Lab Ethics(02)**	**Output/ Result Accuracy (1.5)**	**Documentation & Result Analysis (01)**	**Experiment Total (10)**	**Sign with Date**

Experiment No. 12
Implementation of database connectivity

12.1 Aim: Implementation of database connectivity .

12.2 Course Outcome: Develop database-driven Java applications using JDBC and demonstrate basic understanding of modern frameworks such as SpringBoot.

12.3 Learning Objectives: To develop database driven application.

12.4 Requirement: Java development kit (JDK as per OS) , any IDE NetBeans, eclipse

12.5 Related Theory:

JDBC stands for Java Database Connectivity. It is a standard Java API used to connect Java applications with relational databases such as MySQL, Oracle, PostgreSQL, and SQL Server. JDBC allows a Java program to send SQL queries to a database, retrieve records, insert new data, update existing data, and delete records.

The JDBC API is available in the java.sql package. It provides classes and interfaces required to establish a connection and communicate with the database.

Database connectivity is required when an application needs to store data permanently. Unlike variables and objects, which store data temporarily during program execution, a database stores information permanently and allows it to be retrieved later.

JDBC Architecture

The basic flow of JDBC connectivity is:

Java Application
↓
JDBC API
↓
JDBC Driver
↓
Database

The Java application uses JDBC classes and interfaces. The JDBC driver converts Java database commands into a format understood by the database.

JDBC Driver

A JDBC driver is a software component that enables communication between a Java application and a database. For MySQL, the driver used is called MySQL Connector/J.

The MySQL JDBC driver class is:

com.mysql.cj.jdbc.Driver

In modern JDBC versions, the driver is normally loaded automatically when the Connector/J file is added to the project.

Important JDBC Classes and Interfaces

1. DriverManager

DriverManager manages JDBC drivers and is used to establish a connection with the database.

Connection con = DriverManager.getConnection(url, username, password);

2. Connection

The Connection interface represents an active connection between the Java application and the database.

Connection con;

3. Statement

The Statement interface is used to execute simple SQL statements.

Statement stmt = con.createStatement();

4. PreparedStatement

The PreparedStatement interface is used to execute parameterized SQL queries. It is more secure and efficient than Statement.

PreparedStatement ps = con.prepareStatement(sql);

It helps prevent SQL injection attacks.

5. ResultSet

The ResultSet interface stores the records returned by a SELECT query.

ResultSet rs = stmt.executeQuery(sql);

The cursor of the ResultSet is moved using the next() method.

```
while (rs.next())
{
   System.out.println(rs.getInt(1));
}
```

JDBC URL

The JDBC URL specifies the database type, server address, port number, and database name.

General format:

jdbc:mysql://hostname:port/database_name

Example:

jdbc:mysql://localhost:3306/college

Here:

- jdbc:mysql represents the MySQL JDBC protocol.
- localhost represents the local computer.
- 3306 is the default MySQL port number.
- college is the database name.

Steps for JDBC Connectivity

1. Import the required JDBC package.
2. Load or register the JDBC driver.
3. Establish a connection with the database.
4. Create a Statement or PreparedStatement object.
5. Execute the SQL query.
6. Process the result using ResultSet.
7. Close the database resources.

SQL Execution Methods

The commonly used JDBC execution methods are:

Method	Purpose
executeQuery()	Executes a SELECT query and returns a ResultSet
executeUpdate()	Executes INSERT, UPDATE, or DELETE queries
execute()	Executes general SQL statements

Advantages of JDBC

- Provides connectivity between Java and relational databases.
- Supports different database management systems.
- Allows execution of SQL statements through Java programs.
- Supports insertion, retrieval, updating, and deletion of records.
- Provides transaction management.
- Supports exception handling using SQLException.
- Allows the use of secure parameterized queries through PreparedStatement.

Exception Handling in JDBC

Database operations may generate errors such as an incorrect password, unavailable database server, missing table, or invalid SQL query. JDBC operations are therefore placed inside a try-catch block.

```
catch (SQLException e)
{
   System.out.println(e.getMessage());
}
```

Closing Database Resources

JDBC resources should be closed after use to release memory and database connections.

```
rs.close();
stmt.close();
con.close();
```

The recommended order is to close ResultSet, Statement, and Connection.

12.6 Procedure:

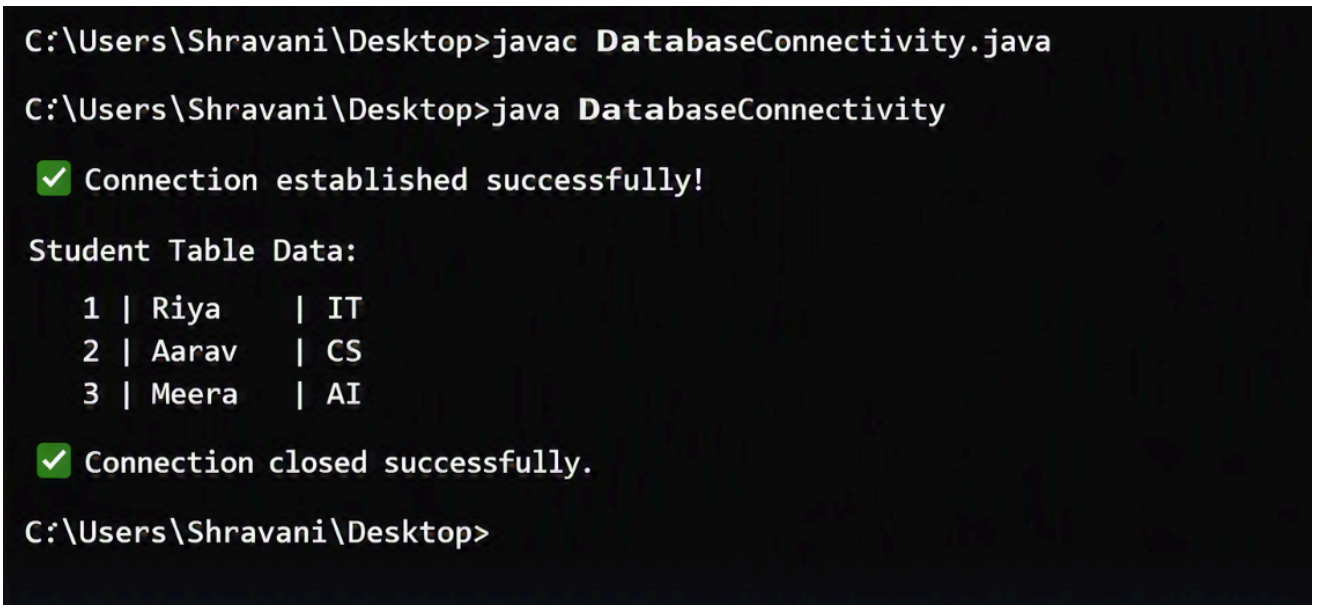
Start MySQL Server and open MySQL Workbench. Create the required database and table using SQL commands. Insert a few sample records into the table. Download MySQL Connector/J and add the connector JAR file to the Java project under Referenced Libraries or include it in the classpath. Create a Java program and import the classes from the java.sql package. Define the JDBC URL, database username, and password. Establish the connection using the DriverManager.getConnection()

method. Create a Statement or PreparedStatement object and execute the required SQL query. If a SELECT query is executed, store the returned records in a ResultSet object and display them using the next() method. Handle database-related errors using a try-catch block. Close the ResultSet, statement, connection, and scanner objects after completing the operation. Compile and run the program and verify that the Java application successfully communicates with the MySQL database.

12.7 Program and Output:

```
import java.sql.*;

public class DatabaseConnectivity {
   public static void main(String[] args) {
      String url = "jdbc:mysql://localhost:3306/college"; // database name: college
      String user = "root"; // your MySQL username
      String password = "1234"; // your MySQL password

      try {
         // 1. Load and register driver (optional in modern JDBC)
         Class.forName("com.mysql.cj.jdbc.Driver");

         // 2. Establish connection
         Connection con = DriverManager.getConnection(url, user, password);
         System.out.println("✅ Connection established successfully!");

         // 3. Create statement
         Statement stmt = con.createStatement();

         // 4. Execute query
         ResultSet rs = stmt.executeQuery("SELECT * FROM student");

         // 5. Process results
         System.out.println("\nStudent Table Data:");
         while (rs.next()) {
            System.out.println(rs.getInt(1) + " | " + rs.getString(2) + " | " + rs.getString(3));
         }
```

output:



12.8 Conclusion:

This experiment demonstrates how JDBC (Java Database Connectivity) enables communication between a Java application and a relational database such as MySQL. By establishing a connection, executing SQL queries, and processing results, we can build dynamic, data-driven applications. The experiment highlights the importance of exception handling and resource management for reliable database operations.

12.9 Questions:

1. What is JDBC?

JDBC stands for Java Database Connectivity — an API that allows Java programs to interact with relational databases using SQL commands.

2. What is the full form of JDBC?

Java Database Connectivity

3. Which package contains the JDBC classes and interfaces?

All JDBC classes and interfaces are contained in the java.sql package.

4. What is the purpose of JDBC?

JDBC provides a standard way for Java applications to connect to databases, execute SQL queries, and manage data (insert, update, delete, retrieve).

5. What is a JDBC driver?

A JDBC driver is a software component that translates Java database calls into database-specific commands, enabling communication between Java and the database.

6. Which JDBC driver is used for MySQL?

The MySQL Connector/J driver — class name:

com.mysql.cj.jdbc.Driver